EXPERIMENT NO. 7

CUSTOMER SEGMENTATION USING K-MEANS CLUSTERING

Name : Harsh Kumar Gola

UID : CU24250165

Course : B.Tech CSE

Section : CSE "B"

Roll No. : 23

AIM

To implement K-Means Clustering on a real-world customer dataset

for customer segmentation and analyze different customer groups

based on demographic and purchasing characteristics.



STEP 1: IMPORT LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

import tkinter as tk
from tkinter import filedialog
```

STEP 2: LOAD DATASET

```
from google.colab import files
import io

uploaded = files.upload()

if not uploaded:
    print("No file selected.")
else:
    for filename in uploaded.keys():
        # Assuming the uploaded file is a CSV
        df = pd.read_csv(io.StringIO(uploaded[filename].decode('utf-8')))
        print("Dataset loaded successfully!")
        print("Shape:", df.shape)
        break # Assuming only one file is uploaded or we take the first one
        df = df.loc[:, ~df.columns.str.contains("^Unnamed")]

# Remove extra spaces from column names
df.columns = df.columns.str.strip()

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

```
Choose Files No file chosen   Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Mall_Customers.csv to Mall_Customers.csv
Dataset loaded successfully!
Shape: (200, 5)
Dataset loaded successfully!
Shape: (200, 5)
Columns: ['CustomerID', 'Genre', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']
```

STEP 3: DATA PREPROCESSING

```
# Remove duplicate records
df = df.drop_duplicates()

# Display missing values
missing_values = df.isnull().sum()

# Fill numerical missing values using median
for column in df.select_dtypes(include=np.number).columns:
    df[column] = df[column].fillna(df[column].median())
```

STEP 4: SELECT FEATURES

```
if {
    "Age",
    "Annual Income (k$)",
    "Spending Score (1-100)"
}.issubset(df.columns):
```

```
    selected_features = [
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]

else:
    print("Required customer features were not found.")
    exit()

X = df[selected_features]

print("Selected features:", selected_features)
```

```
Selected features: ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
```

STEP 5: FEATURE SCALING

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

STEP 6: ELBOW METHOD

```
inertia = []

K_range = range(2, 11)

for k in K_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

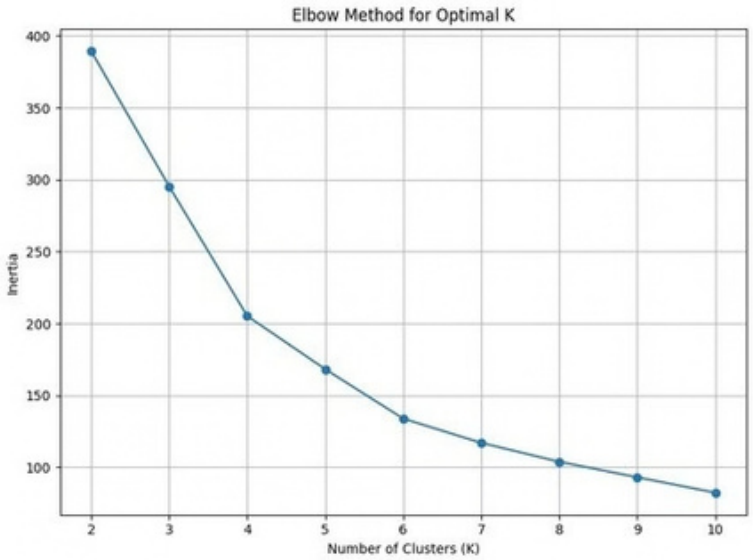
    kmeans.fit(X_scaled)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 6))

plt.plot(
    K_range,
    inertia,
    marker="o"
)

plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(list(K_range))
plt.grid(True)
plt.tight_layout()
plt.show()
```



STEP 7: SILHOUETTE SCORE

```
silhouette_scores = []

for k in K_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)
```

```
    score = silhouette_score(
        X_scaled,
        labels
    )

    silhouette_scores.append(score)

best_k = list(K_range)[
    np.argmax(silhouette_scores)
]

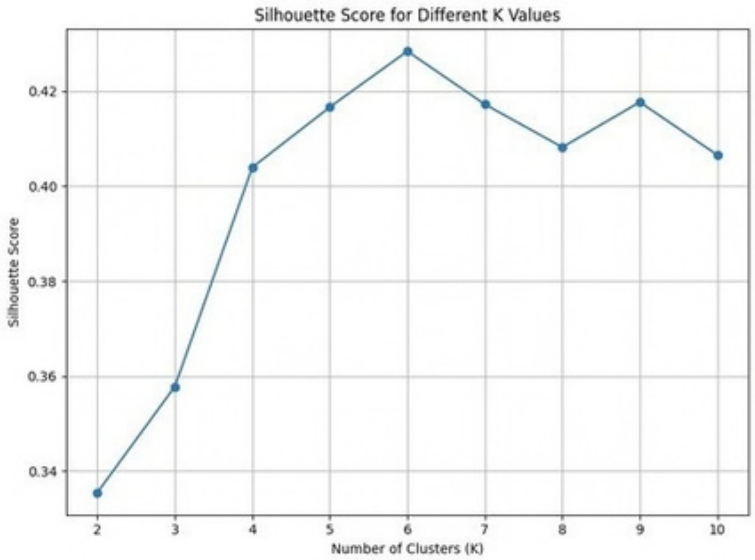
print("Optimal K from Silhouette Score:", best_k)

plt.figure(figsize=(8, 6))

plt.plot(
    K_range,
    silhouette_scores,
    marker="o"
)

plt.title("Silhouette Score for Different K Values")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.xticks(list(K_range))
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Optimal K from Silhouette Score: 6
```



STEP 8: APPLY K-MEANS CLUSTERING

```
kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)
```

STEP 9: CALCULATE FINAL SILHOUETTE SCORE

```
final_silhouette = silhouette_score(
    X_scaled,
    df["Cluster"]
)

print("Final Silhouette Score:", round(final_silhouette, 4))
```

```
Final Silhouette Score: 0.4284
```

STEP 10: CLUSTER VISUALIZATION

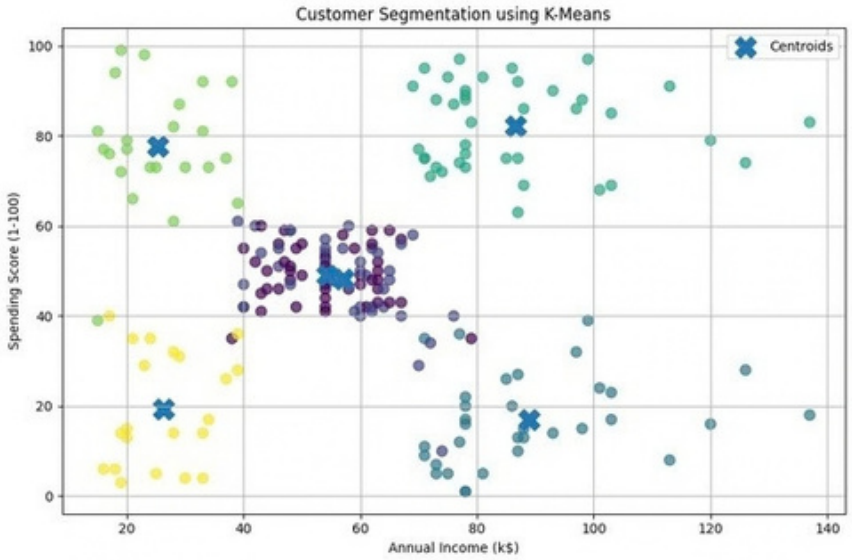
```
plt.figure(figsize=(9, 6))

plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=df["Cluster"],
    s=60,
    alpha=0.7
)

# Convert cluster centers back to original scale
centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)
```

```
plt.scatter(
    centroids[:,  1],
    centroids[:,  2],
    marker="X",
    s=250,
    label="Centroids"
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

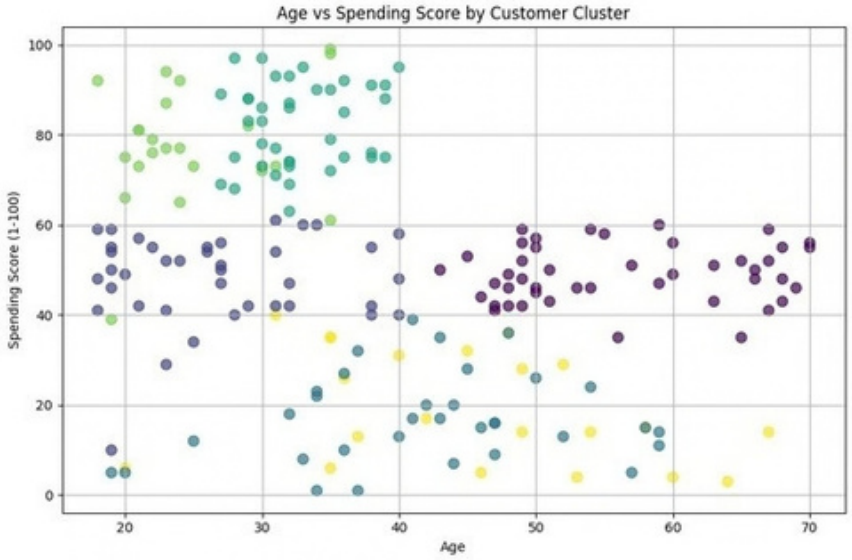


STEP 11: AGE VS SPENDING SCORE

```
plt.figure(figsize=(9, 6))

plt.scatter(
    df["Age"],
    df["Spending Score (1-100)"],
    c=df["Cluster"],
    s=60,
    alpha=0.7
)

plt.title("Age vs Spending Score by Customer Cluster")
plt.xlabel("Age")
plt.ylabel("Spending Score (1-100)")
plt.grid(True)
plt.tight_layout()
plt.show()
```



STEP 12: CUSTOMER CLUSTER PROFILE

```
cluster_profile = df.groupby("Cluster")[
    [
        "Age",
        "Annual Income (k$)",
```

```
        "Spending Score (1-100)"
    ]
].mean()

cluster_size = df["Cluster"].value_counts().sort_index()

print("\nCustomer Cluster Profile:")
print(cluster_profile.round(2))

print("\nCustomers in Each Cluster:")
print(cluster_size)
```

```
Customer Cluster Profile:
          Age Annual Income (k$)  Spending Score (1-100)
Cluster
0       56.33              54.27                   49.07
1       26.79              57.10                   48.13
2       41.94              88.94                   16.97
3       32.69              86.54                   82.13
4       25.00              25.26                   77.61
5       45.52              26.29                   19.38

Customers in Each Cluster:
Cluster
0    45
1    39
2    33
3    39
4    23
5    21

Name: count, dtype: int64
```

STEP 13: CLUSTER CENTROIDS

```
centroid_df = pd.DataFrame(
    centroids,
    columns=selected_features
)

print("\nCluster Centroids:")
print(centroid_df.round(2))
```

```
Cluster Centroids:
      Age Annual Income (k$)  Spending Score (1-100)
0 56.33 1              54.27                   49.07
26.79   2              57.10                   48.13
41.94   3              88.94                   16.97
32.69   4              86.54                   82.13
25.00   5              25.26                   77.61
45.52                  26.29                   19.38
```

STEP 14: IDENTIFY CUSTOMER SEGMENTS

```
for cluster in sorted(df["Cluster"].unique()):

    age = cluster_profile.loc[
        cluster,
        "Age"
    ]

    income = cluster_profile.loc[
        cluster,
        "Annual Income (k$)"
    ]

    spending = cluster_profile.loc[
        cluster,
        "SpendingScore (1-100)"
    ]

    print(
        f"Cluster {cluster}: "
        f"Age={age:.2f}, "
        f"Income={income:.2f}, "
        f"Spending Score={spending:.2f}"
    )
```

```
Cluster 0: Age=56.33, Income=54.27, Spending Score=49.07
Cluster 1: Age=26.79, Income=57.10, Spending Score=48.13
Cluster 2: Age=41.94, Income=88.94, Spending Score=16.97
Cluster 3: Age=32.69, Income=86.54, Spending Score=82.13
Cluster 4: Age=25.00, Income=25.26, Spending Score=77.61
Cluster 5: Age=45.52, Income=26.29, Spending Score=19.38
```

Questions and Answers – Experiment 7

QUESTIONS AND ANSWERS

Q1. What is clustering? How does it differ from classification?

Answer:

Clustering is an unsupervised machine learning technique used to group similar data points into clusters.

Classification is a supervised machine learning technique where the model learns from labelled data and assigns new observations to predefined classes.

Clustering does not require predefined class labels, whereas classification requires labelled training data.

Q2. Explain the working principle of the K-Means Clustering algorithm.

Answer:

K-Means divides data into K clusters.

The basic steps are:

1. Select the number of clusters K.

2. Initialize K cluster centroids.
3. Assign each data point to the nearest centroid.
4. Calculate new centroids based on the assigned points.
5. Repeat the assignment and centroid calculation steps.
6. Stop when the centroids become stable or the algorithm reaches its stopping condition.

The objective is to minimize the distance between data points and their corresponding cluster centroids.

Q3. Why is feature scaling important before applying K-Means clustering?

Answer:

K-Means uses distance calculations to assign data points to clusters.

If features have different scales, a feature with larger values can dominate the distance calculation.

Feature scaling puts variables on a comparable scale so that each feature contributes more fairly to clustering.

StandardScaler was used in this experiment for feature scaling.

Q4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

Answer:

The Elbow Method calculates the within-cluster sum of squares, represented by inertia, for different values of K.

The values of K are plotted against inertia.

As K increases, inertia decreases.

The point where the decrease starts becoming less significant is called the elbow point.

This point provides a useful estimate of the optimal number of clusters.

Q5. What is the Silhouette Score? How is it used to evaluate clustering performance?

Answer:

The Silhouette Score measures how well each data point fits within its assigned cluster compared with other clusters.

A higher Silhouette Score generally indicates better-separated and more compact clusters.

The score ranges approximately from -1 to 1.

A value close to 1 indicates well-separated clusters, while values close to 0 indicate overlapping clusters.

Q6. Why is K-Means considered an unsupervised machine learning algorithm?

Answer:

K-Means is considered unsupervised because it works without predefined target labels.

The algorithm discovers groups or patterns directly from the input data.

In customer segmentation, customers are grouped according to their characteristics without knowing their segment in advance.

Q7. Mention any four real-world applications of customer segmentation.

Answer:

1. Targeted marketing campaigns
2. Personalized product recommendations
3. Customer retention strategies
4. Customer loyalty programs

Other applications include pricing strategies, personalized advertising and market analysis.

Q8. What are the limitations of the K-Means algorithm?

Answer:

1. The number of clusters K generally needs to be selected beforehand.
2. Results can be affected by the initial centroid selection.
3. K-Means can be sensitive to outliers.
4. It works best when clusters are relatively compact and well separated.
5. It may not perform well for clusters with complex shapes.
6. Feature scaling is important because K-Means uses distances.

Q9. How can businesses use customer segmentation to improve marketing and customer retention?

Answer:

Customer segmentation allows businesses to identify groups of customers with similar characteristics and behaviors.

Businesses can create personalized marketing campaigns for different groups.

High-spending customers can receive loyalty rewards.

Low-spending customers can receive special offers.

Potential customers can receive targeted advertisements.

This improves customer engagement, satisfaction and retention.

Q10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

Answer:

K-Means:

- Divides data into a predefined number of clusters.
- Uses cluster centroids.
- Iteratively assigns points to the nearest centroid.
- Generally efficient for large datasets.
- Suitable for customer segmentation and market analysis.

Hierarchical Clustering:

- Builds a hierarchy of clusters.
- Can be agglomerative or divisive.
- Does not require the final number of clusters at the beginning in the same way as K-Means.
- Results can be visualized using a dendrogram.
- Useful for exploring relationships and hierarchical groups.

K-Means is generally preferred when the number of clusters and large-scale segmentation are important, while Hierarchical Clustering is useful when the hierarchy among groups is also important.

